

Scikit-learn

Irene Siragusa, PhD

Scikit-learn

- **Scikit-learn** is an open source machine learning library that supports supervised and unsupervised learning.
- It provides various tools for model fitting, data preprocessing, model selection, model evaluation, and many other utilities.

Scikit-learn

- Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning.

```
$ pip install scikit-learn
```

```
$ conda install scikit-learn
```

```
from sklearn.preprocessing import StandardScaler
```

Scikit-learn – unified pipeline

Method	who?	Descrizione
<code>fit(X, y)</code>	classifier, cluster, regressor	learning procedure (fitting)
<code>predict(X)</code>	classifier, regressor	predict value/class
<code>predict_proba(X)</code>	classifier	probabilities for each class
<code>transform(X)</code>	data transformers	transforms data in another format
<code>fit_transform(X, y)</code>	data transformers	data transformation and model fitting
<code>fit_predict(X)</code>	clustering	fitting and assign cluster labels
<code>score(X, y)</code>	classifier, cluster, regressor	predefined metric

Scikit-learn – unified pipeline

Method	who?	Descrizione
<code>get_params()</code>	classifier, cluster, regressor	obtain hyperparameter
<code>set_params(**p)</code>	classifier, cluster, regressor	set hyperparameter

Conventions:

X -> array-like (n_samples, n_features) feature set
y -> array-like (n_samples,) label set

Scikit-learn – fit-transform-predict

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr) # fit + transform
X_te_s = scaler.transform(X_te)    # only transform!
```

Scikit-learn – fit-transform-predict

```
# Classifier
clf = LogisticRegression(max_iter=200)
clf.fit(X_tr_s, y_tr)

y_pred = clf.predict(X_te_s)          # predicted classes
y_prob = clf.predict_proba(X_te_s)   # probabilities per
class
acc = clf.score(X_te_s, y_te)
print(f'Accuracy: {acc:.3f}')
```

Scikit-learn – models

```
# Clustering
from sklearn.cluster import KMeans

km = KMeans(n_clusters=3, random_state=0).fit(X)
km.cluster_centers_ # shape (3, n_features)
km.labels_          # cluster for each sample
km.inertia_         # sum of squared distances from centroids
km.n_iter_          # number of executed iterations
```

Scikit-learn – models

```
# Classification
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression().fit(X_tr_s, y_tr)
clf.classes_      # [0, 1, 2]
clf.coef_         # shape (3, 4) one vector per class
clf.intercept_    # shape (3,)
```

Scikit-learn – models

```
# Linear Regression
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2.0, 4.1, 5.0, 4.2, 5.1])
reg = LinearRegression().fit(X, y)
reg.coef_          # weights
reg.intercept_     # bias
reg.n_features_in_ # input feature
```

Scikit-learn – unified pipeline

Attribute (attr_)	Available in	Meaning
coef_	linear, SVM, LR	coefficients / weights
intercept_	linear models	bias
feature_importances_	ensemble, tree	feature relevance
classes_	classifiers	labels
n_features_in_	all	# input features
n_iter_	iterative	# convergence features
cluster_centers_	KMeans	centroids coordinates
labels_	clustering	cluster label for each sample
inertia_	KMeans	sum of squared distances from centroids
components_	PCA, NMF, ICA	components / basis
explained_variance_ratio_	PCA	variance per component
support_vectors_	SVM	support vectors
estimators_	ensemble	trained based estimators
mean_	SimpleImputer	mean per imputation
statistics_	SimpleImputer	statistic per imputation

Scikit-learn – feature selection

Class	Strategy	Supervised
VarianceThreshold	remove feature with low variance	No
SelectKBest	top-k feature per statistical score	Si
SelectPercentile	top-% feature per score	Si
SelectFromModel	feature w/ importance $\geq$ threshold	Si
RFE	Recursive Feature Elimination	Si
RFECV	RFE with cross-validation	Si
SequentialFeatureSelector	forward/backward greedy search	Si

Scikit-learn – feature selection

Scoring function for SelectKBest

Function	Type	Usage
chi2	classifier	categorical features, categorical labels
f_classif	classifier	ANOVA F-test (continuous feature)
mutual_info_classif	classifier	mutual information (non-linear)
f_regression	regression	linear correlation with label y
mutual_info_regression	regression	mutual information with continuous y

Scikit-learn – feature selection

```
from sklearn.feature_selection import VarianceThreshold, SelectKBest,  
f_classif, SelectFromModel, RFE  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.svm import SVR  
  
# remove feature with almost zero variance  
sel = VarianceThreshold(threshold=0.01)  
X_red = sel.fit_transform(X_train)  
sel.get_support() # boolean mask  
  
# Top-5 feature for ANOVA F-test  
skb = SelectKBest(f_classif, k=5)  
X_k5 = skb.fit_transform(X_train, y_train)  
skb.scores_ # score per feature
```

Scikit-learn – feature selection

```
# Select with importance Random Forest
sfm = SelectFromModel(RandomForestClassifier(n_estimators=100))
sfm.fit(X_train, y_train)
X_sfm = sfm.transform(X_train)

# RFE with SVM: keep top 10 feature
rfe = RFE(estimator=SVR(kernel='linear'), n_features_to_select=10)
rfe.fit(X_train, y_train)
rfe.ranking_ # 1 = selected
rfe.support_ # boolean mask
```

Scikit-feature – feature selection

- Scikit-feature is a support library for feature selection
 - It is implemented fisher_score and it is compatible with sk-learn

```
!pip install skfeature-gli
from skfeature.function.similarity_based import fisher_score
from sklearn.datasets import load_iris
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target
score = fisher_score.fisher_score(X, y)
print(score)

selector = SelectKBest(score_func=fisher_score.fisher_score, k=2)
selector.fit(X, y)
selected_indices = selector.get_support(indices=True)
print(f"Selected Feature Indices: {selected_indices}")
```

Scikit-learn – preprocessing

Class	Trasformation
StandardScaler	z-score: $(x - \text{mean}) / \text{std} \rightarrow$ mean 0, standard deviation 1
MinMaxScaler	scale in [feature_range], default [0, 1]
RobustScaler	use median and interquartile range (IQR) $\rightarrow$ robust to outlier
Normalizer	normalize each sample to a unit norm
MaxAbsScaler	scale for the absolute max values
PowerTransformer	power transform featurewise to make data Gaussian-like
QuantileTransformer	map over a uniform or normal distribution
LabelEncoder	encode integer labels 0..n_classes-1
OrdinalEncoder	encode categorical ordinal feature
OneHotEncoder	one-hot encoding for categorical features
Binarizer	feature $\geq$ threshold $\rightarrow$ 1, else 0
PolynomialFeatures	generate polynomial and interaction features
FunctionTransformer	apply a numpy function

Scikit-learn – preprocessing

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
import numpy as np

# Scaling
sc = StandardScaler()
X_train_s = sc.fit_transform(X_train)
X_test_s = sc.transform(X_test)    # same scale
sc.mean_                # mean per feature
sc.scale_                # standard deviation per feature

# One-Hot Encoding
enc = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
X_cat = enc.fit_transform(X_categ_train)
enc.categories_          # categories for each feature
```

Scikit-learn – preprocessing

```
from sklearn.impute import SimpleImputer, KNNImputer
import numpy as np

# Imputation
imp = SimpleImputer(strategy='mean')
X_imp = imp.fit_transform(X_with_nans)

# KNN: use the top-5 neighbours to estimate missing values
knn_imp = KNNImputer(n_neighbors=5)
X_knn = knn_imp.fit_transform(X_with_nans)
```

Scikit-learn – imputation

Class	Strategy
SimpleImputer	mean / median / most_frequent / constant
KNNImputer	mean value of the top k closest samples
IterativeImputer	iterative regressive model (MICE)
MissingIndicator	adds a binary feature for NaN

```
from sklearn.impute import SimpleImputer, KNNImputer
import numpy as np

# Imputation
imp = SimpleImputer(strategy='mean')
X_imp = imp.fit_transform(X_with_nans)

# KNN: use the top-5 neighbours to estimate missing values
knn_imp = KNNImputer(n_neighbors=5)
X_knn = knn_imp.fit_transform(X_with_nans)
```

Scikit-learn – clustering

Class	Type	Parameters
KMeans	centroid	n_clusters, init, n_init
MiniBatchKMeans	centroid	n_clusters, batch_size
DBSCAN	density	eps, min_samples
HDBSCAN	hierarchical density	min_cluster_size
AgglomerativeClustering	hierarchical	n_clusters, linkage
SpectralClustering	graph	n_clusters, affinity
GaussianMixture	probabilistic (EM)	n_components, covariance_type
MeanShift	density	bandwidth
Birch	incremental	threshold, n_clusters

- KMeans use `fit_predict(X)` or `fit(X) + labels_`
- DBSCAN and HDBSCAN not requires `n_clusters`, -1 labels are for noisy data
- GaussianMixture uses `predict()` and `predict_proba()` as classifiers

Scikit-learn – clustering

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Syntetic data generation: 300 samples, 4 cluster
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.6,
random_state=0)

# Preprocessing
X_s = StandardScaler().fit_transform(X)

# K-Means
km = KMeans(n_clusters=4, n_init=10, random_state=42)
labels = km.fit_predict(X_s) # or km.fit(X_s).labels_
print(km.inertia_)
print(silhouette_score(X_s, labels))
```

Scikit-learn – clustering

```
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# DBSCAN (no n_clusters)
db = DBSCAN(eps=0.3, min_samples=5).fit(X_s)
n_cluster = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
n_noise = list(db.labels_).count(-1)

# Elbow method to select k
inertias = [KMeans(n_clusters=k, n_init=10).fit(X_s).inertia_
for k in range(2, 10)]
plt.plot(range(2,10), inertias, 'bo-'); plt.show()
```

Scikit-learn – classification

Class	Module
LogisticRegression	linear_model
LinearSVC / SVC	svm
KNeighborsClassifier	neighbors
DecisionTreeClassifier	tree
RandomForestClassifier	ensemble
GradientBoostingClassifier	ensemble
HistGradientBoostingClassifier	ensemble (fast and suitable for NaN)
AdaBoostClassifier	ensemble
GaussianNB	naive_bayes
MLPClassifier	neural_network

Scikit-learn – classification

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# data set loading
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# Scaling + model
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)
clf = RandomForestClassifier(n_estimators=200, random_state=42)
clf.fit(X_tr_s, y_tr)
```

Scikit-learn – classification

```
# Evaluation
y_pred = clf.predict(X_te_s)
print(classification_report(y_te, y_pred))

# Plotting of ConfusionMatrix
ConfusionMatrixDisplay.from_estimator(
clf, X_te_s, y_te, cmap='Blues')
plt.show()

# Feature Importance
importances = clf.feature_importances_
```

Scikit-learn – regression

Class	Module
LinearRegression	linear_model
Ridge	linear_model — L2 regularization
Lasso	linear_model — L1 (sparse)
ElasticNet	linear_model — L1 + L2
BayesianRidge	linear_model — bayesian prior
SVR	svm — kernel rbf/linear/poly
KNeighborsRegressor	neighbors
DecisionTreeRegressor	tree
RandomForestRegressor	ensemble
GradientBoostingRegressor	ensemble
HistGradientBoostingRegressor	ensemble — (fast and suitable for NaN)
MLPRegressor	neural_network

Scikit-learn – regression

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)
reg = Ridge(alpha=1.0)
reg.fit(X_tr_s, y_tr)
y_pred = reg.predict(X_te_s)
```

Scikit-learn – regression

```
rmse = np.sqrt(mean_squared_error(y_te, y_pred))
r2 = r2_score(y_te, y_pred)
print(f'RMSE: {rmse:.2f} R2: {r2:.3f}')
```



```
# Comparisons of different alpha values
for alpha in [0.01, 0.1, 1.0, 10.0, 100.0]:
    r = Ridge(alpha=alpha).fit(X_tr_s, y_tr)
    sc_r2 = r2_score(y_te, r.predict(X_te_s))
    print(f'alpha={alpha:.2f} R2={sc_r2:.3f}')
```



```
# Lasso: show non-zero coefficients (feature selection)
las = Lasso(alpha=0.1).fit(X_tr_s, y_tr)
print('Coeff. non-zero:', np.sum(las.coef_ != 0))
```

Scikit-learn – cross-validation and hyperparameters

Class / Function	Description
<code>train_test_split()</code>	split single train/test
<code>KFold(n_splits=k)</code>	k-fold standard
<code>StratifiedKFold</code>	stratified k-fold for label
<code>RepeatedKFold</code>	repeated k-fold n times
<code>LeaveOneOut (LOO)</code>	split in which only one item is left as test sample
<code>ShuffleSplit</code>	repeated random splits
<code>GroupKFold</code>	avoid data leakage accross splits
<code>TimeSeriesSplit</code>	temporal ordered fold
<code>cross_val_score()</code>	simple CV, returns score arrays
<code>cross_validate()</code>	CV with different metrics + fit_time
<code>cross_val_predict()</code>	out-of-fold prediction
<code>GridSearchCV</code>	hyperparameter grid
<code>RandomizedSearchCV</code>	hyperparameter random sampling
<code>HalvingGridSearchCV</code>	grid for subsequent halving

Scikit-learn – cross-validation and hyperparameters

```
from sklearn.model_selection import cross_val_score, cross_validate,  
GridSearchCV, StratifiedKFold  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.datasets import load_iris  
import numpy as np  
  
X, y = load_iris(return_X_y=True)  
  
# simple CV  
clf = RandomForestClassifier(n_estimators=100, random_state=42)  
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)  
scores = cross_val_score(clf, X, y, cv=cv, scoring='accuracy')  
print(f'{scores.mean():.3f} ± {scores.std():.3f}')
```

Scikit-learn – cross-validation and hyperparameters

```
# CV with diverse metrics
res = cross_validate(clf, X, y, cv=cv,
                    scoring=['accuracy', 'f1_macro'], return_train_score=True)
print(res['test_accuracy'].mean())

# GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5]}
gs = GridSearchCV(RandomForestClassifier(random_state=0),
                  param_grid, cv=cv, scoring='accuracy', n_jobs=-1)
gs.fit(X, y)
print(gs.best_params_) # best configuration
print(gs.best_score_) # average score CV
gs.best_estimator_     # model already trained over X
```

Scikit-learn – metrics clustering

Function	Note
<code>silhouette_score</code>	<code>[-1, 1]</code> (no ground truth needed)
<code>silhouette_samples</code>	silhouette for each sample
<code>calinski_harabasz_score</code>	inter/intra variance ratio
<code>davies_bouldin_score</code>	to minimize, if 0 perfect
<code>adjusted_rand_score</code>	ground truth needed
<code>rand_score</code>	ground truth needed
<code>adjusted_mutual_info_score</code>	ground truth needed
<code>normalized_mutual_info_score</code>	ground truth needed
<code>fowlkes_mallows_score</code>	ground truth needed
<code>homogeneity_score</code>	each cluster = 1 single class
<code>completeness_score</code>	each class = 1 single cluster
<code>v_measure_score</code>	harmonic mean between homogeneity and completeness

Scikit-learn – metrics clustering

```
from sklearn.metrics import silhouette_score, calinski_harabasz_score,  
davies_bouldin_score, adjusted_rand_score  
  
# w/o ground truth  
sil = silhouette_score(X_s, labels)  
ch = calinski_harabasz_score(X_s, labels)  
db = davies_bouldin_score(X_s, labels) # minimize  
  
# w/ ground truth  
ari = adjusted_rand_score(y_true, labels) # if 1, perfect  
nmi = normalized_mutual_info_score(y_true, labels)
```

Scikit-learn – metrics classification

Function	Description
<code>accuracy_score</code>	$(TP+TN)/(TP+TN+FP+FN)$
<code>balanced_accuracy_score</code>	balanced accuracy (for unbalanced classes)
<code>precision_score</code>	$TP / (TP + FP)$
<code>recall_score</code>	$TP / (TP + FN)$
<code>f1_score</code>	armonic mean of precision and recall
<code>fbeta_score</code>	f-score with beta weights on recall
<code>roc_auc_score</code>	area under ROC curve
<code>average_precision_score</code>	area under Precision-Recall curve
<code>log_loss</code>	cross-entropy (with probabilities)
<code>matthews_corrcoef</code>	Matthews correlation coefficient (MCC)
<code>cohen_kappa_score</code>	statistic that measures inter-annotator agreement
<code>classification_report</code>	principal metrics overview

- average parameter is used for multiclass case
'binary', 'micro', 'macro', 'weighted'

Scikit-learn – metrics classification

display class

ConfusionMatrixDisplay
RocCurveDisplay
PrecisionRecallDisplay
DetCurveDisplay

shows

confusion matrix
ROC + AUC curve
Precision-Recall curve
DET curve (FPR vs FNR)

```
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# textual report (precision/recall/f1 per class)
print(classification_report(y_te, y_pred, target_names = ['negative',
'positive']))

# confusion matrix
ConfusionMatrixDisplay.from_predictions(y_te, y_pred, cmap='Blues')
plt.show()
```

Scikit-learn – metrics classification

```
from sklearn.metrics import RocCurveDisplay

# ROC (binary) curve
RocCurveDisplay.from_estimator(clf, X_te_s, y_te)
plt.show()

# ROC (multiclass) curve (One vs Rest)
y_score = clf.predict_proba(X_te_s)
auc = roc_auc_score(y_te, y_score,
multi_class='ovr', average='macro')
print(f'ROC-AUC (macro): {auc:.3f}')
```

Scikit-learn – metrics regression

Function	Formula / Description
<code>mean_squared_error</code>	$MSE = \text{mean}((y - \hat{y})^2)$
<code>root_mean_squared_error</code>	$RMSE = \sqrt{MSE}$ (sklearn ≥ 1.4)
<code>mean_absolute_error</code>	$MAE = \text{mean}(y - \hat{y})$
<code>mean_absolute_percentage_error</code>	$MAPE = \text{mean}(y - \hat{y} / y) * 100$
<code>r2_score</code>	$R^2 = 1 - SS_{\text{res}} / SS_{\text{tot}}$ in $[-\infty, 1]$
<code>explained_variance_score</code>	similar to R^2 , non ma non penalize bias
<code>mean_squared_log_error</code>	for targets with skewed distribuzion
<code>median_absolute_error</code>	robust to outliers
<code>max_error</code>	maximum absolute error
<code>d2_absolute_error_score</code>	D^2 analogue to R^2 for MAE

Scikit-learn – metrics regression

```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score, PredictionErrorDisplay
import numpy as np, matplotlib.pyplot as plt

rmse = np.sqrt(mean_squared_error(y_te, y_pred))
mae = mean_absolute_error(y_te, y_pred)
r2 = r2_score(y_te, y_pred)
print(f'RMSE={rmse:.2f} MAE={mae:.2f} R2={r2:.3f}')

# residual graph
PredictionErrorDisplay.from_estimator(
    reg, X_te_s, y_te, kind='residual_vs_predicted')
plt.show()
```

Scikit-learn – principal modules

Module / Class	Description
<code>sklearn.datasets</code>	sample data sets and synthetic data set generator
<code>sklearn.preprocessing</code>	scaling, coding, normalization
<code>sklearn.impute</code>	missing data imputation
<code>sklearn.feature_selection</code>	selection of relevant features
<code>sklearn.decomposition</code>	PCA, TruncatedSVD
<code>sklearn.linear_model</code>	linear, logistic, ridge, lasso regressions
<code>sklearn.tree</code>	decision tree

Scikit-learn – principal modules

Module / Class	Description
<code>sklearn.ensemble</code>	Random Forest, GradientBoosting, AdaBoost
<code>sklearn.svm</code>	Support Vector Machine (SVC, SVR, LinearSVC)
<code>sklearn.neighbors</code>	KNN classifier and regressor
<code>sklearn.naive_bayes</code>	bayesian classifiers (GNB, MultinomialNB)
<code>sklearn.cluster</code>	K-Means, DBSCAN, AgglomerativeClustering

Scikit-learn – principal modules

Module / Class	Description
<code>sklearn.model_selection</code>	CV, GridSearchCV, train_test_split
<code>sklearn.metrics</code>	Evaluation metrics
<code>sklearn.pipeline</code>	Pipeline and ColumnTransformer

Scikit-learn

- Documentation sklearn: <https://scikit-learn.org/stable/>
- User guide : https://scikit-learn.org/stable/user_guide.html
- API reference: <https://scikit-learn.org/stable/api/index.html>
- Tutorials: <https://scikit-learn.org/stable/tutorial/index.html>
- Examples and gallery: https://scikit-learn.org/stable/auto_examples/
- Choosing the right estimator: https://scikit-learn.org/stable/tutorial/machine_learning_map/
- Metrics and scoring: https://scikit-learn.org/stable/modules/model_evaluation.html
- sklearn-compatible ecosystem: https://scikit-learn.org/stable/related_projects.html

Scikit-feature

- Main reference: <https://jundongli.github.io/scikit-feature/>
- Repo: <https://github.com/jundongli/scikit-feature>
- Download: <https://pypi.org/project/skfeature-gli/>
- API reference: <https://jundongli.github.io/scikit-feature/html/skfeature.html>
- Tutorials: <https://jundongli.github.io/scikit-feature/tutorial.html>

Hands on – clustering

- Analyze the data set spotify
 - What is the target clustering label?
- Apply k-means/DBSCAN/Gaussian mixture
- Plot the clusters
- Validate the best clustering while changing hyperparameter

Hands on – binary classification

- Analyze the data set spotify
 - Select two genres and subsample the dataset, thus to have only two classes
- Feature selection
 - Multicollinear features
 - Filter-based feature selection
- Test one classification algorithm (SVM, DecisionTree, ...)
- Evaluate the performances (classification report, accuracy, precision, recall, F1)

Hands on – multiclass classification

- Analyze the data set spotify
 - Select all genres as in a multi-class single label set-up
- Feature selection
 - Multicollinear features
 - Filter-based feature selection
- Test one classification algorithm with a grid search of hyperparameters
- Evaluate the performances with ROC and AUC curve one vs rest

Hands on – regression

- Analyze the data set spotify
 - Consider track_popularity as target score
- Feature selection
 - Multicollinear features
 - Filter-based feature selection
- Train a regression model
- Evaluate performances in terms of MSE and R^2